

Package ‘TRexSelector’

May 18, 2026

Type Package

Title T-Rex Selector Variable Selection Framework

Version 0.1.0

Author Fabian Scheidt

Maintainer Fabian Scheidt <fabian.scheidt@example.com>

Description FDR-controlled variable selection for regression and classification, providing bindings to the TRexSelector C++ library.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

LinkingTo Rcpp, RcppEigen, BH, Rcereal

Imports Rcpp, R6

SystemRequirements C++20

NeedsCompilation yes

Contents

agglomerative_cluster	2
as.matrix.mmap_matrix	3
compute_fdp	3
compute_fdp_dense	4
compute_precision	4
compute_recall	5
compute_tpp	5
compute_tpp_dense	6
convert_to_memory_mapped	6
cut_tree	7
dim.mmap_matrix	7
DistanceMetric	8
LinkageMethod	8
LpNormScaler	9
mmap_matrix	11
print.mmap_matrix	12
RidgeCV	12

RidgeGCV	14
TRexBiobankScreeningSelector	16
TRexDASelector	18
TRexGVSSelector	20
TRexScreeningSelector	22
TRexSelector	23
TSolver	25
ZScoreScaler	27
[.mmap_matrix	30
Index	31

agglomerative_cluster *Agglomerative Clustering*

Description

Performs hierarchical agglomerative clustering on a numeric matrix.

Usage

```
agglomerative_cluster(
  data,
  method = LinkageMethod$Ward,
  metric = DistanceMetric$Euclidean,
  use_mmap = FALSE
)
```

Arguments

data	A numeric matrix.
method	A value from LinkageMethod (e.g. LinkageMethod\$Ward).
metric	A value from DistanceMetric (e.g. DistanceMetric\$Euclidean). Default is Euclidean.
use_mmap	Whether to use memory mapping files internally. Default is FALSE.

Value

A numeric matrix representing the computed hierarchical linkage.

as.matrix.mmap_matrix *Get Zero-Copy View as Numeric Vector (Block)*

Description

Get Zero-Copy View as Numeric Vector (Block)

Usage

```
## S3 method for class 'mmap_matrix'
as.matrix(x, ...)
```

Arguments

x	mmap_matrix object
...	extra arguments including row_range and col_range

Value

Numeric matrix (copied)

compute_fdp	<i>Compute False Discovery Proportion (FDP) using index sets</i>
-------------	--

Description

Compute False Discovery Proportion (FDP) using index sets

Usage

```
compute_fdp(selected_indices, true_support)
```

Arguments

selected_indices	Integer vector of selected indices.
true_support	Integer vector of true active indices.

Value

Numeric FDP value.

compute_fdp_dense	<i>Compute False Discovery Proportion (FDP) using dense vectors</i>
-------------------	---

Description

Compute False Discovery Proportion (FDP) using dense vectors

Usage

```
compute_fdp_dense(beta_hat, beta, eps = 1e-15)
```

Arguments

beta_hat	Numeric vector of estimated coefficients.
beta	Numeric vector of true coefficients.
eps	Numeric threshold for counting non-zeros (default: 1e-15).

Value

Numeric FDP value.

compute_precision	<i>Compute Precision using index sets</i>
-------------------	---

Description

Compute Precision using index sets

Usage

```
compute_precision(selected_indices, true_support)
```

Arguments

selected_indices	Integer vector of selected indices.
true_support	Integer vector of true active indices.

Value

Numeric precision value.

compute_recall	<i>Compute Recall using index sets</i>
----------------	--

Description

Compute Recall using index sets

Usage

```
compute_recall(selected_indices, true_support)
```

Arguments

selected_indices	Integer vector of selected indices.
true_support	Integer vector of true active indices.

Value

Numeric recall value.

compute_tpp	<i>Compute True Positive Proportion (TPP) using index sets</i>
-------------	--

Description

Compute True Positive Proportion (TPP) using index sets

Usage

```
compute_tpp(selected_indices, true_support)
```

Arguments

selected_indices	Integer vector of selected indices.
true_support	Integer vector of true active indices.

Value

Numeric TPP value.

compute_tpp_dense	<i>Compute True Positive Proportion (TPP) using dense vectors</i>
-------------------	---

Description

Compute True Positive Proportion (TPP) using dense vectors

Usage

```
compute_tpp_dense(beta_hat, beta, eps = 1e-15)
```

Arguments

beta_hat	Numeric vector of estimated coefficients.
beta	Numeric vector of true coefficients.
eps	Numeric threshold for counting non-zeros (default: 1e-15).

Value

Numeric TPP value.

convert_to_memory_mapped	<i>Convert R Matrix to System Memory Mapped Matrix</i>
--------------------------	--

Description

Maps a given R numeric matrix into a binary formatted memory-mapped file on disk efficiently.

Usage

```
convert_to_memory_mapped(mat, filename)
```

Arguments

mat	A numeric matrix.
filename	String destination file path.

Value

NULL invisibly mapped.

`cut_tree`*Cut Hierarchical Linkage Matrix*

Description

Flatten the dendrogram clustering results based on the number of requested groups.

Usage

```
cut_tree(linkage, num_orig_objs, num_clusters)
```

Arguments

<code>linkage</code>	A numeric matrix representing linkage, generated by <code>agglomerative_cluster</code> .
<code>num_orig_objs</code>	Original number of observations data rows.
<code>num_clusters</code>	Target flat group capacity.

Value

An integer vector mapping observation indexes to cluster arrays.

`dim.mmap_matrix`*Get mmap_matrix dimensions*

Description

Get `mmap_matrix` dimensions

Usage

```
## S3 method for class 'mmap_matrix'  
dim(x)
```

Arguments

<code>x</code>	<code>mmap_matrix</code> object
----------------	---------------------------------

Value

an integer vector of length 2

DistanceMetric	<i>Distance Metrics</i>
----------------	-------------------------

Description

Enumeration of supported distance metrics for hierarchical clustering. Supported metrics include: - Euclidean: Standard Euclidean distance: codes as 0. - Correlation: Correlation distance (1 - correlation): codes as 1. - Manhattan: Manhattan (L1) distance: codes as 2. - Correlation_LSH_Filter: Correlation distance with LSH-based filtering: codes as 3. - Correlation_LSH_Approx: Approximate correlation distance using LSH: codes as 4.

Usage

DistanceMetric

Format

An object of class list of length 5.

LinkageMethod	<i>Linkage Methods</i>
---------------	------------------------

Description

Enumeration of supported hierarchical clustering linkage methods. Supported methods include: - Ward: Ward's minimum variance method: codes as 0. - Average (UPGMA): Average linkage: codes as 1. - Complete: Complete linkage: codes as 2. - Single: Single linkage: codes as 3. - WPGMA: Weighted pair group method with arithmetic mean: codes as 4. - Median: Median linkage: codes as 5. - Centroid: Centroid linkage: codes as 6.

Usage

LinkageMethod

Format

An object of class list of length 7.

LpNormScaler

*Lp-Norm Scaler***Description**

Scales input features to unit norm based on the chosen L1 or L2 formulation.

Methods**Public methods:**

- `LpNormScaler$new()`
- `LpNormScaler$fit()`
- `LpNormScaler$transform_inplace()`
- `LpNormScaler$inverse_transform_inplace()`
- `LpNormScaler$is_fitted()`
- `LpNormScaler$get_dropped_indices()`
- `LpNormScaler$get_means()`
- `LpNormScaler$get_scales()`
- `LpNormScaler$get_with_mean()`
- `LpNormScaler$get_with_norm()`
- `LpNormScaler$get_norm_type()`
- `LpNormScaler$save()`
- `LpNormScaler$load()`
- `LpNormScaler$clone()`

Method `new()`: Initialize the LpNormScaler

Usage:

```
LpNormScaler$new(norm_type = 2, with_mean = TRUE)
```

Arguments:

`norm_type` Integer, 1 for L1 norm, 2 for L2 norm (default: 2)

`with_mean` Logical, whether to center the data before scaling (default: TRUE)

Method `fit()`: Compute the norm parameters.

Usage:

```
LpNormScaler$fit(X, threshold = 1e-12)
```

Arguments:

`X` A numeric matrix.

`threshold` Double, lower bound for norm (default: 1e-12).

Returns: Value.

Method `transform_inplace()`: Perform standardization in-place.

Usage:

```
LpNormScaler$transform_inplace(X)
```

Arguments:

`X` A numeric matrix.

Returns: The modified matrix X .

Method `inverse_transform_inplace()`: Inverse transform to original scale.

Usage:

```
LpNormScaler$inverse_transform_inplace(X)
```

Arguments:

X A numeric matrix.

Returns: The modified matrix X .

Method `is_fitted()`: Check if scaler is fitted.

Usage:

```
LpNormScaler$is_fitted()
```

Returns: Value.

Method `get_dropped_indices()`: Get dropped indices.

Usage:

```
LpNormScaler$get_dropped_indices()
```

Returns: Value.

Method `get_means()`: Get the learned means.

Usage:

```
LpNormScaler$get_means()
```

Returns: Value.

Method `get_scales()`: Get the learned scales.

Usage:

```
LpNormScaler$get_scales()
```

Returns: Value.

Method `get_with_mean()`: Check if scaler centers data.

Usage:

```
LpNormScaler$get_with_mean()
```

Returns: Value.

Method `get_with_norm()`: Check if scaler scales data to unit norm.

Usage:

```
LpNormScaler$get_with_norm()
```

Returns: Value.

Method `get_norm_type()`: Get norm type (1 for L1, 2 for L2).

Usage:

```
LpNormScaler$get_norm_type()
```

Returns: Value.

Method `save()`: Save model state to file.

Usage:

```
LpNormScaler$save(filename)
```

Arguments:

filename String path to save output.

Returns: Value.

Method load(): Load model state from file.

Usage:

LpNormScaler\$load(filename)

Arguments:

filename String path to load input from.

Returns: Value.

Method clone(): The objects of this class are cloneable with this method.

Usage:

LpNormScaler\$clone(deep = FALSE)

Arguments:

deep Whether to make a deep clone.

mmap_matrix

Memory-Mapped Matrix Object

Description

S3 object wrapping an external pointer to a C++ MemoryMappedMatrix.

Usage

```
mmap_matrix(filename, rows, cols, mode = "readwrite")
```

Arguments

filename	Path to binary mmap file
rows	Number of rows
cols	Number of columns
mode	Access mode: 'readonly' or 'readwrite' (default)

Value

Object of class 'mmap_matrix' (S3 XPtr wrapper)

<code>print.mmap_matrix</code>	<i>Print mmap_matrix</i>
--------------------------------	--------------------------

Description

Print mmap_matrix

Usage

```
## S3 method for class 'mmap_matrix'
print(x, ...)
```

Arguments

<code>x</code>	mmap_matrix object
<code>...</code>	arguments passed to other methods

Value

invisibly returns x

RidgeCV	<i>Ridge Regression with Cross-Validation</i>
---------	---

Description

Performs explicit K-Fold cross-validation for Ridge Regression model selection.

Methods**Public methods:**

- `RidgeCV$new()`
- `RidgeCV$fit()`
- `RidgeCV$cv_min()`
- `RidgeCV$cv_1se()`
- `RidgeCV$index_min()`
- `RidgeCV$index_1se()`
- `RidgeCV$get_lambdas()`
- `RidgeCV$get_cv_errors()`
- `RidgeCV$get_cv_std()`
- `RidgeCV$clone()`

Method `new()`: Initialize the RidgeCV object.

Usage:

```
RidgeCV$new()
```

Method `fit()`: Fit the CV model.

Usage:

```
RidgeCV$fit(  
  X,  
  y,  
  num_folds = 10,  
  n_lambda = 100,  
  lambda_ratio = 1000,  
  seed = 0  
)
```

Arguments:

X Predictor matrix.

y Response vector.

num_folds Number of folds (default 10).

n_lambda Number of lambdas in path (default 100).

lambda_ratio Ratio of max to min lambda (default 1000).

seed Random seed for CV splits (default 0).

Returns: Value.

Method cv_min(): Get lambda that minimizes CV error.

Usage:

```
RidgeCV$cv_min()
```

Returns: Value.

Method cv_1se(): Get 1SE rule lambda.

Usage:

```
RidgeCV$cv_1se()
```

Returns: Value.

Method index_min(): Get index of minimum CV error.

Usage:

```
RidgeCV$index_min()
```

Returns: Value.

Method index_1se(): Get index of 1SE rule lambda.

Usage:

```
RidgeCV$index_1se()
```

Returns: Value.

Method get_lambdas(): Get the sequence of lambdas used.

Usage:

```
RidgeCV$get_lambdas()
```

Returns: Value.

Method get_cv_errors(): Get the CV mean squared errors for each lambda.

Usage:

```
RidgeCV$get_cv_errors()
```

Returns: Value.

Method `get_cv_std()`: Get the standard error of the CV errors.

Usage:

`RidgeCV$get_cv_std()`

Returns: Value.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

`RidgeCV$clone(deep = FALSE)`

Arguments:

`deep` Whether to make a deep clone.

RidgeGCV

Ridge Regression with GCV

Description

Efficiently solves Ridge Regression while computing Generalized Cross-Validation (GCV).

Methods

Public methods:

- `RidgeGCV$new()`
- `RidgeGCV$fit()`
- `RidgeGCV$solve()`
- `RidgeGCV$gcv()`
- `RidgeGCV$predict()`
- `RidgeGCV$predict_path()`
- `RidgeGCV$solve_path()`
- `RidgeGCV$gcv_optimal()`
- `RidgeGCV$default_lambda_grid()`
- `RidgeGCV$n_samples()`
- `RidgeGCV$n_features()`
- `RidgeGCV$rank()`
- `RidgeGCV$clone()`

Method `new()`: Initialize the RidgeGCV object.

Usage:

`RidgeGCV$new()`

Method `fit()`: Fit the SVD decomposition for the given data.

Usage:

`RidgeGCV$fit(X, y)`

Arguments:

`X` Predictor matrix.

`y` Response vector.

Returns: Value.

Method `solve()`: Solve for a specific lambda.

Usage:

`RidgeGCV$solve(lambda_val)`

Arguments:

`lambda_val` Numeric value for lambda.

Returns: Value.

Method `gcv()`: Compute GCV score for a specific lambda.

Usage:

`RidgeGCV$gcv(lambda_val)`

Arguments:

`lambda_val` Numeric value for lambda.

Returns: Value.

Method `predict()`: Predict using a specific lambda.

Usage:

`RidgeGCV$predict(X_new, lambda_val)`

Arguments:

`X_new` Numeric matrix for predictions.

`lambda_val` Numeric value for lambda.

Returns: Value.

Method `predict_path()`: Predict using a path of lambdas.

Usage:

`RidgeGCV$predict_path(X_new, lambdas)`

Arguments:

`X_new` Numeric matrix for predictions.

`lambdas` Numeric vector of lambda values.

Returns: Value.

Method `solve_path()`: Solve across a path of lambdas.

Usage:

`RidgeGCV$solve_path(lambdas)`

Arguments:

`lambdas` Numeric vector of lambda values.

Returns: Value.

Method `gcv_optimal()`: Find the optimal lambda.

Usage:

`RidgeGCV$gcv_optimal(num_grid = 200, tol = 1e-08)`

Arguments:

`num_grid` Integer, number of grid points.

`tol` Numeric, tolerance for convergence.

Returns: Value.

Method `default_lambda_grid()`: Get the default lambda grid.

Usage:

```
RidgeGCV$default_lambda_grid(num_grid = 100, ratio = 1000)
```

Arguments:

`num_grid` Integer, number of grid points.

`ratio` Numeric, ratio for lambda grid.

Returns: Value.

Method `n_samples()`: Get number of samples.

Usage:

```
RidgeGCV$n_samples()
```

Returns: Value.

Method `n_features()`: Get number of features.

Usage:

```
RidgeGCV$n_features()
```

Returns: Value.

Method `rank()`: Get rank of X.

Usage:

```
RidgeGCV$rank()
```

Returns: Value.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
RidgeGCV$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

TRexBiobankScreeningSelector

Biobank Screen T-Rex Selector

Description

Genomic biobank phenotype screening algorithm based on TRex architecture.

Methods

Public methods:

- `TRexBiobankScreeningSelector$new()`
- `TRexBiobankScreeningSelector$select()`
- `TRexBiobankScreeningSelector$clone()`

Method `new()`: Create a new `TRexBiobankScreeningSelector` object.

Usage:

```
TRexBiobankScreeningSelector$new(
  X,
  Y,
  target_FDR_trex = 0.1,
  lower_bound_FDR = 0.05,
  upper_bound_FDR = 0.15,
  use_bootstrap_CI = FALSE,
  R_boot = 1000,
  ci_grid_step = 0.001,
  trex_method = "TREX",
  method = "LARS",
  use_omp = TRUE,
  use_memory_mapping = FALSE,
  max_outer_threads = 1,
  max_inner_threads = 1,
  seed = -1,
  verbose = TRUE
)
```

Arguments:

X Feature matrix.

Y Response vector (1D) or matrix (2D).

target_FDR_trex Target FDR for fallback T-Rex.

lower_bound_FDR Lower bound for acceptable estimated FDR.

upper_bound_FDR Upper bound for acceptable estimated FDR.

use_bootstrap_CI Whether to use bootstrap CI screening.

R_boot Number of bootstrap iterations.

ci_grid_step Grid step sizes for interval calculations.

trex_method Screening method: "TREX", "TREX_DA_AR1", "TREX_DA_EQUI", "TREX_DA_BLOCK_EQUI".

method The solver method, e.g., "LARS", "TACGP", "TENET" (default: "LARS").

use_omp Whether to use OpenMP parallelization (default: TRUE).

use_memory_mapping Use out-of-core memory mapping (default: FALSE).

max_outer_threads Thread count for K repetitions (default: 1).

max_inner_threads Thread count for inner solvers (default: 1).

seed Random seed (default: -1).

verbose Whether to print progress (default: TRUE).

Method `select()`: Run the Biobank Screening algorithm.

Usage:

```
TRexBiobankScreeningSelector$select()
```

Returns: A list with results. For vector 'Y', it gives a single phenotype result. For matrix 'Y', it gives a multi-phenotype result containing a \$statistics data.frame and \$selected_indices arrays.

Method clone(): The objects of this class are cloneable with this method.

Usage:

```
TRexBiobankScreeningSelector$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TRexDASelector

Dependency-Aware T-Rex Selector (TRexDA)

Description

FDR-controlled variable selection algorithm that handles strong correlations.

Methods

Public methods:

- [TRexDASelector\\$new\(\)](#)
- [TRexDASelector\\$select\(\)](#)
- [TRexDASelector\\$get_selected_indices\(\)](#)
- [TRexDASelector\\$get_results\(\)](#)
- [TRexDASelector\\$get_matrices\(\)](#)
- [TRexDASelector\\$clone\(\)](#)

Method new(): Create a new TRexDASelector object.

Usage:

```
TRexDASelector$new(
  X,
  y,
  tFDR = 0.1,
  seed = -1,
  verbose = TRUE,
  method = "LARS",
  da_method = "BT",
  cor_coef = -2,
  rho_thr_DA = 0.02,
  hc_linkage = "Single",
  hc_grid_length = 0,
  K = 20,
  max_dummy_multiplier = 10,
  use_max_T_stop = TRUE,
  opt_threshold = 0.75,
  lloop_strategy = "HCONCAT",
  tloop_stagnation_stop = TRUE,
  tloop_max_stagnant_steps = 5,
  use_omp = TRUE,
```

```

    use_memory_mapping = FALSE,
    max_outer_threads = 1,
    max_inner_threads = 1,
    tol = 1e-04,
    lambda2 = 0.1,
    rho_afs = 0.3,
    ncgmp_variant = 0
)

```

Arguments:

X Feature matrix or MemoryMappedMatrix.

y Response vector.

tFDR Target FDR level (default: 0.1).

seed Random seed (default: -1).

verbose Whether to print progress (default: TRUE).

method The solver method, e.g., "LARS", "TACGP", "TENET" (default: "LARS").

da_method Dependency method: "BT", "AR1", "EQUI" (default: "BT").

cor_coef Correlation coefficient for AR1/EQUI.

rho_thr_DA Threshold below which equi-correlation is skipped (default: 0.02).

hc_linkage Hierarchical clustering linkage: "Single", "Complete", "Average", "WPGMA", "Ward" (default: "Single").

hc_grid_length Number of grid points for BT (default: 0).

K Number of random experiments (default: 20).

max_dummy_multiplier Max dummy features multiplier (default: 10).

use_max_T_stop Whether to bound max T_stop (default: TRUE).

opt_threshold Optimization threshold (default: 0.75).

lloop_strategy Strategy for L-loop (default: "HCONCAT").

tloop_stagnation_stop Whether to stop early if stagnation (default: TRUE).

tloop_max_stagnant_steps Steps before stagnation trigger (default: 5).

use_omp Whether to use OpenMP parallelization (default: TRUE).

use_memory_mapping Use out-of-core memory mapping (default: FALSE).

max_outer_threads Thread count for K repetitions (default: 1).

max_inner_threads Thread count for inner solvers (default: 1).

tol Tolerance for solver algorithms (default: 1e-4).

lambda2 ENET hyperparameter (default: 0.1).

rho_afs AFS hyperparameter (default: 0.3).

ncgmp_variant NCGMP strategy (default: 0).

Method `select()`: Run the Dependency-Aware TRexSelector algorithm.

Usage:

```
TRexDASelector$select()
```

Method `get_selected_indices()`: Get the integer indices (1-based) of the selected variables.

Usage:

```
TRexDASelector$get_selected_indices()
```

Returns: Numeric vector of selected indices.

Method `get_results()`: Get generic solver results.

Usage:

```
TRexDASelector$get_results()
```

Returns: A list containing results.

Method `get_matrices()`: Get DA diagnostic matrices.

Usage:

```
TRexDASelector$get_matrices()
```

Returns: A list containing structured matrices.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
TRexDASelector$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

TRexGVSSelector

T-Rex Selector with Group Variable Selection (TRex GVS)

Description

FDR-controlled variable selection algorithm utilizing group-based structural augmentation.

Methods

Public methods:

- `TRexGVSSelector$new()`
- `TRexGVSSelector$select()`
- `TRexGVSSelector$get_selected_indices()`
- `TRexGVSSelector$get_results()`
- `TRexGVSSelector$clone()`

Method `new()`: Create a new TRexGVSSelector object.

Usage:

```
TRexGVSSelector$new(
  X,
  y,
  tFDR = 0.1,
  seed = -1,
  verbose = TRUE,
  method = "LARS",
  gvs_type = "EN",
  corr_max = 0.5,
  hc_linkage = "Single",
  lambda_2 = 0,
  groups = NULL,
  K = 20,
  max_dummy_multiplier = 10,
  use_max_T_stop = TRUE,
```

```

    opt_threshold = 0.75,
    lloop_strategy = "HCONCAT",
    tloop_stagnation_stop = TRUE,
    tloop_max_stagnant_steps = 5,
    use_omp = TRUE,
    use_memory_mapping = FALSE,
    max_outer_threads = 1,
    max_inner_threads = 1
  )

```

Arguments:

X Feature matrix or MemoryMappedMatrix.

y Response vector.

tFDR Target FDR level (default: 0.1).

seed Random seed (default: -1).

verbose Whether to print progress (default: TRUE).

method The solver method (default: "LARS").

gvs_type Augmentation policy: "EN", "IEN", "GL", "IGL", "SGL", "ISGL" (default: "EN").

corr_max Maximum pairwise correlation for clustering (default: 0.5).

hc_linkage Hierarchical clustering linkage method: "Single", "Complete", "Average", "WPGMA" (default: "Single").

lambda_2 Lambda2 for grouping.

groups User-specified groups (default: NULL).

K Number of random experiments (default: 20).

max_dummy_multiplier Max dummy features multiplier (default: 10).

use_max_T_stop Whether to bound max T_stop (default: TRUE).

opt_threshold Optimization threshold (default: 0.75).

lloop_strategy Strategy for L-loop (default: "HCONCAT").

tloop_stagnation_stop Whether to stop early if stagnation (default: TRUE).

tloop_max_stagnant_steps Steps before stagnation trigger (default: 5).

use_omp Whether to use OpenMP parallelization (default: TRUE).

use_memory_mapping Use out-of-core memory mapping (default: FALSE).

max_outer_threads Thread count for K repetitions (default: 1).

max_inner_threads Thread count for inner solvers (default: 1).

Method `select()`: Run the GVS algorithm.

Usage:

```
TRexGVSSelector$select()
```

Method `get_selected_indices()`: Get the integer indices (1-based) of the selected variables.

Usage:

```
TRexGVSSelector$get_selected_indices()
```

Returns: Numeric vector of selected indices.

Method `get_results()`: Get generic solver results.

Usage:

```
TRexGVSSelector$get_results()
```

Returns: A list containing results.

Method clone(): The objects of this class are cloneable with this method.

Usage:

```
TRexGVSSelector$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TRexScreeningSelector *Screen T-Rex Selector*

Description

Fast variable screening algorithm based on TRex architecture.

Methods

Public methods:

- [TRexScreeningSelector\\$new\(\)](#)
- [TRexScreeningSelector\\$select\(\)](#)
- [TRexScreeningSelector\\$get_selected_indices\(\)](#)
- [TRexScreeningSelector\\$get_results\(\)](#)
- [TRexScreeningSelector\\$get_matrices\(\)](#)
- [TRexScreeningSelector\\$clone\(\)](#)

Method new(): Create a new TRexScreeningSelector object.

Usage:

```
TRexScreeningSelector$new(
  X,
  y,
  seed = -1,
  verbose = TRUE,
  use_bootstrap_CI = FALSE,
  R_boot = 1000,
  ci_grid_step = 0.001,
  trex_method = "TRES",
  method = "LARS",
  use_omp = TRUE,
  use_memory_mapping = FALSE,
  max_outer_threads = 1,
  max_inner_threads = 1
)
```

Arguments:

X Feature matrix or MemoryMappedMatrix.

y Response vector.

seed Random seed (default: -1).

verbose Whether to print progress (default: TRUE).

use_bootstrap_CI Whether to use bootstrap CI screening.

R_boot Number of bootstrap iterations.

`ci_grid_step` the grid step sizes for interval calculations
`trex_method` Screening method: "TREX", "TREX_DA_AR1", "TREX_DA_EQUI", "TREX_DA_BLOCK_EQUI"
`method` The solver method, e.g., "LARS", "TACGP", "TENET" (default: "LARS").
`use_omp` Whether to use OpenMP parallelization (default: TRUE).
`use_memory_mapping` Use out-of-core memory mapping (default: FALSE).
`max_outer_threads` Thread count for K repetitions (default: 1).
`max_inner_threads` Thread count for inner solvers (default: 1).

Method `select()`: Run the Screening algorithm.

Usage:

```
TRexScreeningSelector$select()
```

Method `get_selected_indices()`: Get the integer indices (1-based) of the selected variables.

Usage:

```
TRexScreeningSelector$get_selected_indices()
```

Returns: Numeric vector of selected indices.

Method `get_results()`: Get generic solver results.

Usage:

```
TRexScreeningSelector$get_results()
```

Returns: A list containing results.

Method `get_matrices()`: Get solver matrices.

Usage:

```
TRexScreeningSelector$get_matrices()
```

Returns: A list containing matrices.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
TRexScreeningSelector$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

 TRexSelector

T-Rex Selector

Description

FDR-controlled variable selection algorithm.

Methods

Public methods:

- `TRexSelector$new()`
- `TRexSelector$select()`
- `TRexSelector$clone()`

Method `new()`: Create a new TRexSelector object.

Usage:

```
TRexSelector$new(
  X,
  y,
  tFDR = 0.1,
  seed = -1,
  verbose = TRUE,
  method = "LARS",
  K = 20,
  max_dummy_multiplier = 10,
  use_max_T_stop = TRUE,
  opt_threshold = 0.75,
  lloop_strategy = "HCONCAT",
  tloop_stagnation_stop = TRUE,
  tloop_max_stagnant_steps = 5,
  use_omp = TRUE,
  use_memory_mapping = FALSE,
  max_outer_threads = 1,
  max_inner_threads = 1,
  tol = 1e-04,
  lambda2 = 0.1,
  rho_afs = 0.3,
  ncgmp_variant = 0
)
```

Arguments:

`X` Feature matrix or MemoryMappedMatrix.

`y` Response vector.

`tFDR` Target FDR level (default: 0.1).

`seed` Random seed (default: -1).

`verbose` Whether to print progress (default: TRUE).

`method` The solver method, e.g., "LARS", "TACGP", "TENET" (default: "LARS").

`K` Number of random experiments (default: 20).

`max_dummy_multiplier` Max dummy features multiplier (default: 10).

`use_max_T_stop` Whether to bound max T_stop (default: TRUE).

`opt_threshold` Optimization threshold (default: 0.75).

`llloop_strategy` Strategy for L-loop (default: "HCONCAT").

`tloop_stagnation_stop` Whether to stop early if stagnation (default: TRUE).

`tloop_max_stagnant_steps` Steps before stagnation trigger (default: 5).

`use_omp` Whether to use OpenMP parallelization (default: TRUE).

`use_memory_mapping` Use out-of-core memory mapping (default: FALSE).

`max_outer_threads` Thread count for K repetitions (default: 1).

max_inner_threads Thread count for inner solvers (default: 1).
 tol Tolerance for solver algorithms (default: 1e-4).
 lambda2 ENET hyperparameter (default: 0.1).
 rho_afs AFS hyperparameter (default: 0.3).
 ncgmp_variant NCGMP strategy (default: 0).

Method `select()`: Run the TRexSelector algorithm.

Usage:

`TRexSelector$select()`

Returns: A list containing results including `selected_var`, `v_thresh`, and `selected_indices`.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

`TRexSelector$clone(deep = FALSE)`

Arguments:

`deep` Whether to make a deep clone.

TSolver

Base T-Solver Class

Description

Wraps the underlying C++ TSolver instances over R6.

Public fields

`X` Numeric matrix of predictors.
`D` Numeric matrix of dummies/features.
`y` Numeric vector of response variable.
`ptr` External pointer to C++ object.

Methods

Public methods:

- `TSolver$new()`
- `TSolver$execute_step()`
- `TSolver$get_beta()`
- `TSolver$get_cp()`
- `TSolver$get_rss()`
- `TSolver$get_r2()`
- `TSolver$get_dof()`
- `TSolver$get_actives()`
- `TSolver$get_inactives()`
- `TSolver$get_intercept()`
- `TSolver$clone()`

Method `new()`: Method description.

Usage:

```
TSolver$new(solver_type = "lars", X, D, y)
```

Arguments:

solver_type Character, type of solver to use.

X Numeric matrix of predictors.

D Numeric matrix of dummies/features.

y Numeric vector of response variable.

Method execute_step(): Method description.

Usage:

```
TSolver$execute_step(T_stop = -1, early_stop = TRUE)
```

Arguments:

T_stop Integer, stopping criterion.

early_stop Logical, whether to stop early.

Returns: Value.

Method get_beta(): Method description.

Usage:

```
TSolver$get_beta(step = -1)
```

Arguments:

step Integer, step number.

Returns: Value.

Method get_cp(): Method description.

Usage:

```
TSolver$get_cp()
```

Returns: Value.

Method get_rss(): Method description.

Usage:

```
TSolver$get_rss()
```

Returns: Value.

Method get_r2(): Method description.

Usage:

```
TSolver$get_r2()
```

Returns: Value.

Method get_dof(): Method description.

Usage:

```
TSolver$get_dof()
```

Returns: Value.

Method get_actives(): Method description.

Usage:

```
TSolver$get_actives(r_index = TRUE)
```

Arguments:

`r_index` Integer, starting from 1 in R.

Returns: Value.

Method `get_inactives()`: Method description.

Usage:

`TSolver$get_inactives(r_index = TRUE)`

Arguments:

`r_index` Integer, starting from 1 in R.

Returns: Value.

Method `get_intercept()`: Method description.

Usage:

`TSolver$get_intercept(step = -1)`

Arguments:

`step` Integer, step number.

Returns: Value.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

`TSolver$clone(deep = FALSE)`

Arguments:

`deep` Whether to make a deep clone.

ZScoreScaler

Z-Score Scaler

Description

Standardizes features by removing the mean and scaling to unit variance.

Methods

Public methods:

- `ZScoreScaler$new()`
- `ZScoreScaler$fit()`
- `ZScoreScaler$transform_inplace()`
- `ZScoreScaler$inverse_transform_inplace()`
- `ZScoreScaler$is_fitted()`
- `ZScoreScaler$get_dropped_indices()`
- `ZScoreScaler$get_with_mean()`
- `ZScoreScaler$get_with_std()`
- `ZScoreScaler$get_means()`
- `ZScoreScaler$get_scales()`
- `ZScoreScaler$save()`

- `ZScoreScaler$load()`
- `ZScoreScaler$clone()`

Method `new()`: Initialize the ZScoreScaler

Usage:

```
ZScoreScaler$new(with_mean = TRUE, with_std = TRUE)
```

Arguments:

`with_mean` Logical, whether to center the data before scaling (default: TRUE)

`with_std` Logical, whether to scale the data to unit variance (default: TRUE)

Method `fit()`: Compute the mean and std to be used for later scaling.

Usage:

```
ZScoreScaler$fit(X, threshold = 1e-12)
```

Arguments:

`X` A numeric matrix.

`threshold` Double, lower bound for standard deviation (default: 1e-12).

Returns: Value.

Method `transform_inplace()`: Perform standardization by centering and scaling in-place.

Usage:

```
ZScoreScaler$transform_inplace(X)
```

Arguments:

`X` A numeric matrix.

Returns: The modified matrix `X`.

Method `inverse_transform_inplace()`: Inverse transform to original scale.

Usage:

```
ZScoreScaler$inverse_transform_inplace(X)
```

Arguments:

`X` A numeric matrix.

Returns: The modified matrix `X`.

Method `is_fitted()`: Check if scaler is fitted.

Usage:

```
ZScoreScaler$is_fitted()
```

Returns: Value.

Method `get_dropped_indices()`: Get dropped indices.

Usage:

```
ZScoreScaler$get_dropped_indices()
```

Returns: Value.

Method `get_with_mean()`: Check if scaler centers data.

Usage:

```
ZScoreScaler$get_with_mean()
```

Returns: Value.

Method `get_with_std()`: Check if scaler scales data to unit variance.

Usage:

```
ZScoreScaler$get_with_std()
```

Returns: Value.

Method `get_means()`: Get the learned means.

Usage:

```
ZScoreScaler$get_means()
```

Returns: Value.

Method `get_scales()`: Get the learned scales (standard deviations).

Usage:

```
ZScoreScaler$get_scales()
```

Returns: Value.

Method `save()`: Save model state to file.

Usage:

```
ZScoreScaler$save(filename)
```

Arguments:

`filename` String path to save output.

Returns: Value.

Method `load()`: Load model state from file.

Usage:

```
ZScoreScaler$load(filename)
```

Arguments:

`filename` String path to load input from.

Returns: Value.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
ZScoreScaler$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

[.mmap_matrix	<i>Extract block from mmap_matrix</i>
---------------	---------------------------------------

Description

Extract block from mmap_matrix

Usage

```
## S3 method for class 'mmap_matrix'  
x[i, j, ..., drop = FALSE]
```

Arguments

x	mmap_matrix object
i	row indices
j	column indices
...	unused arguments
drop	logical. If TRUE the result is coerced to the lowest possible dimension (not supported).

Value

a matrix block

Index

- * **datasets**
 - DistanceMetric, [8](#)
 - LinkageMethod, [8](#)
 - [.mmap_matrix, [30](#)
- agglomerative_cluster, [2](#)
- as.matrix.mmap_matrix, [3](#)
- compute_fdp, [3](#)
- compute_fdp_dense, [4](#)
- compute_precision, [4](#)
- compute_recall, [5](#)
- compute_tpp, [5](#)
- compute_tpp_dense, [6](#)
- convert_to_memory_mapped, [6](#)
- cut_tree, [7](#)
- dim.mmap_matrix, [7](#)
- DistanceMetric, [8](#)
- LinkageMethod, [8](#)
- LpNormScaler, [9](#)
- mmap_matrix, [11](#)
- print.mmap_matrix, [12](#)
- RidgeCV, [12](#)
- RidgeGCV, [14](#)
- TRexBiobankScreeningSelector, [16](#)
- TRexDASelector, [18](#)
- TRexGVSSelector, [20](#)
- TRexScreeningSelector, [22](#)
- TRexSelector, [23](#)
- TSolver, [25](#)
- ZScoreScaler, [27](#)